



C Code Optimization on *SHARC ADSP-21489*

In paving the way for quality in DSP (Digital Signal Processing) applications, most of the time people end up in computational constraints, especially in embedded world. Hence optimization becomes the commensurate stint to development. This paper will be focusing on computationally optimizing C applications on SHARC ADSP-21489, one of the most powerful DSP processors developed by Analog Devices. In this whitepaper, we will discuss general optimizations like compiler options, intrinsics etc., and also the effective SHARC hardware usage like available memory-to-memory transfers and accelerators which have bigger impact on optimizations.

Read on.

Authored by: Sravan Kumar
Mahantesh Belakhindi

Introduction

SHARC processors have a sophisticated signal processing functionality in a multitude of applications. The processors' feature-rich core and peripherals have made it a logical choice for product developers. But for an effective usage of this resource, it's imperative for programmers to learn about its architecture, compiler settings and other library resources for these processors. This whitepaper focuses on computational optimization of SHARC ADSP-21489 processor using CCES (CrossCore Embedded Studio) IDE. In the following sections we will discuss:

- Compiler settings which generate the assembly code
- Analog Devices which provide architecture-specific optimized DSP libraries
- Examples on how to computationally optimize application using hardware resources like FFT (Fast Fourier Transform)/IIR (Infinite Impulse Response)/FIR (Finite Impulse Response) accelerators, DMAs and memories

Note: Discussion on memory constraints is out of the scope of this whitepaper.

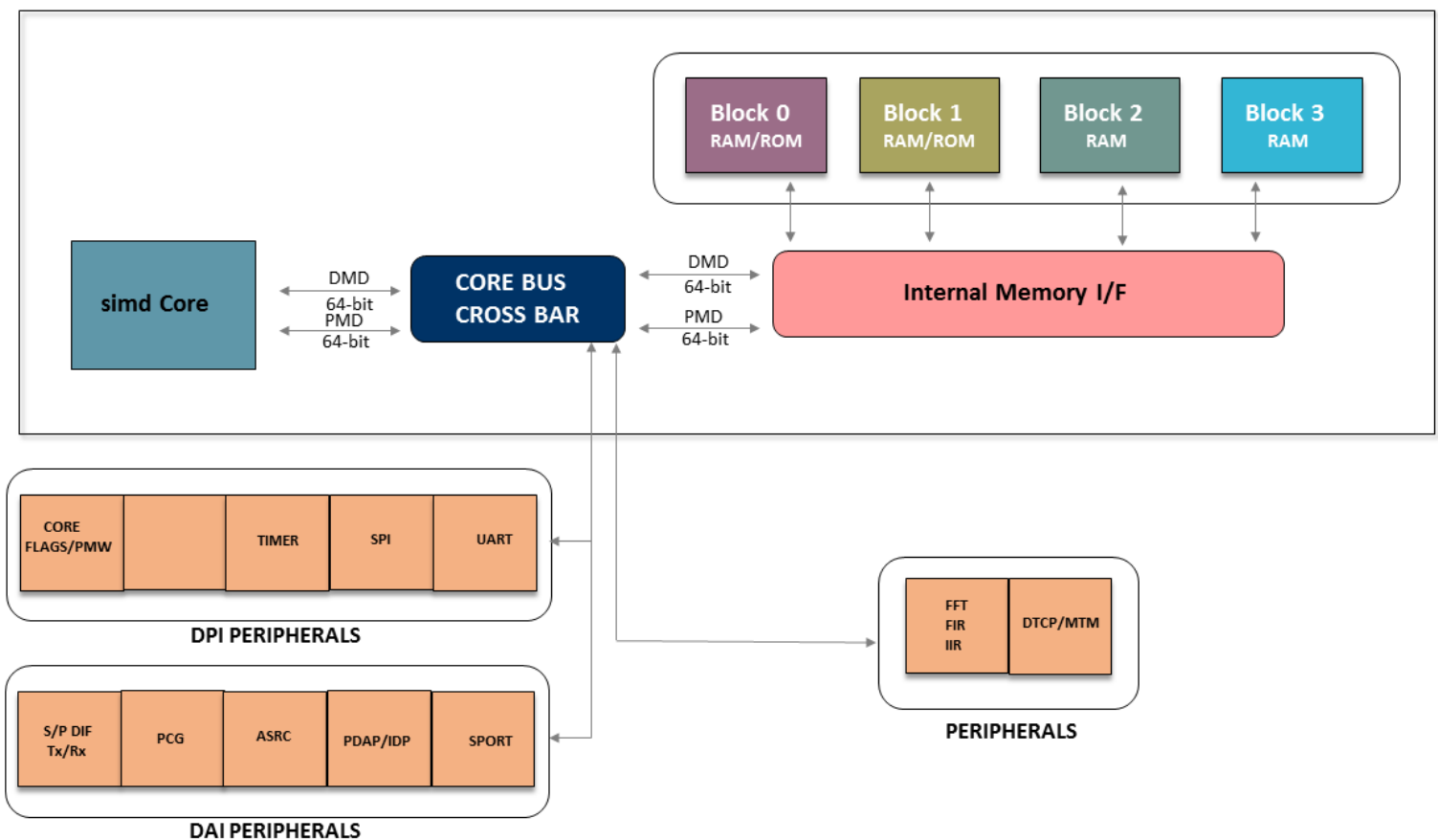


Figure 1. Architecture Overview of ADSP-21489

Architecture Introduction

The most important point for optimizing is to write a code suited to the architecture. Hence, the user is required to have the knowledge of the target architecture to efficiently generate the optimized code.

In ADSP-SHARC 21489 processor, SHARC stands for **S**uper **H**arvard **A**rchitecture (SHARC). This four bus architecture has two address buses and two data buses, but provides a single, unified address space for program and data storage. While the data memory (DM) bus only carries data, the program memory (PM) bus handles instructions & data, thus allowing a dual-data access.

It has 8 SPORTS, UART, GPIO, DAI, Timer, Watchdog timer etc. as other DSPs along with its main specifications. The core processor is a five-stage pipeline. It has two such processing units, enabling Single Instruction Multiple Data (SIMD).

Clock Speed	400 MHz
RAM	5 Mbits (640 KB / 160 KW)
Hardware Accelerators	FIR, FFT, IIR
SIMD	two processing elements (PE _x , PE _y)
DMAs	Internal memory ↔ Internal memory (MTM) Internal memory ↔ External memory Internal memory ↔ Accelerators Internal memory ↔ UART Internal memory ↔ SPI Internal memory ↔ SPORT

Table 1. Capabilities of the ADSP-21489

The discussion will pursue the following order:

compiler settings in brief > introduction to ADSP library > using core & accelerators simultaneously to achieve parallel processing

Compiler Options

Compiler Optimizations: This is the basic level of optimization, where the code size may increase but it results in a significant computational gain when the level of Optimization is set to O3 (max).

Profile Guided Optimization (PGO): PGO is an excellent way to tune the compiler's optimization strategy for the typical run-time behavior of a program. There are many characteristics of a program which cannot be known statically at compile-time but can be provided through PGO. The compiler can use this knowledge to improve its code generation. The benefits include:

- Relatively accurate branch prediction
- Improved loop transformations
- Reduced code size

Intraprocedural Optimization: The optimizer often requires information that can only be determined by looking outside the function that it is optimizing. For example, it helps to know what data can be referenced by pointer parameters or whether a variable actually has a constant value.

In this case the compiler is called again from the link phase to recompile the program using additional information obtained during previous compilations. Using Inline functions, one can restrict keyword for the non-overlapping pointers.

Both index array method and pointer increment method can be employed for accessing arrays. It depends upon how the assembly code is generated. But, one should avoid dereferencing the same pointer multiple times.

Improving Conditional Code: Some built-in functions will help to predict the condition-code i.e. whether it's likely to occur or not. Based on that, the compiler will optimize. This also gives certain computational gains.

(Refer the example in section "Improving Conditional Code" in [Compiler Manual for SHARC Processors](#))

Loop Optimizations: In general, while programming in CCES, avoid loop unrolling and loop rotation as compiler optimization will optimize it effectively and if explicitly done; then compiler can't generate optimized code. And also, conditional loop codes, calling function call inside loops etc. has to be minimized. For efficient code optimization over loops, refer the example in section "Loop optimization Pragmas" from [Compiler Manual for SHARC Processors](#).

Using Built-In Functions: Built-in functions, also known as compiler intrinsics, provide a method for the programmer to efficiently use low-level features of the processor hardware while programming in C language.

Using Pragmas: There are many compiler-specific pragmas which help in loop iterations prediction, alignment prediction etc. to the compiler and helps it in generating optimized code. Refer [Compiler Manual for SHARC Processors](#) for more information.

ADSP - DSP Libraries

DSP library provides many intrinsic functions such as DSP, math, statistical etc. These library functions effectively use the highly optimized hardware resources for the respective SHARC architecture.

The services provided by the DSP run-time library include support for general-purpose signal processing such as compander, filters, windowing functions, FFT functions etc. It supports complex-based math operations too. These services are Analog Devices extensions to ANSI standard C. These functions can be used for SIMD mode operations. Refer "DSP Run-Time Library section" in [SHARC Library Manual](#) for CCES.

Effectively Using Architecture

Using different memories: Since SHARC supports dual-memory access, if any operation is performed on two arrays, try placing these arrays in different block locations. With SIMD, fetching dual data at a time is computationally very effective.

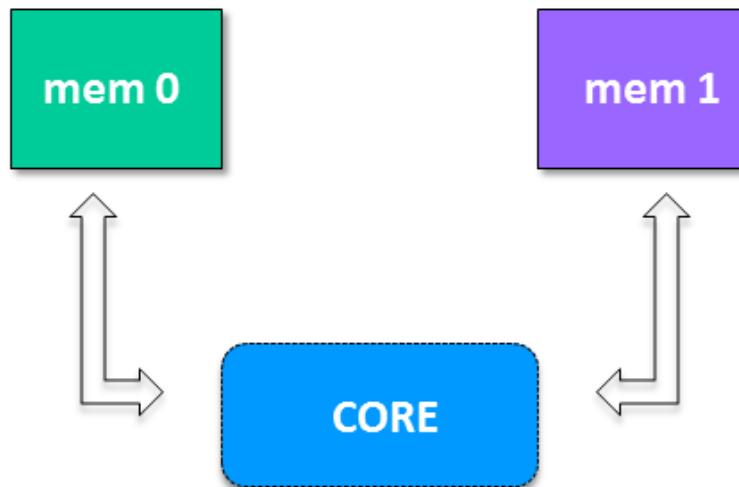


Figure 2. Data Fetching from two Memory Blocks simultaneously

Linker Description File (.ldf) will give the overall information of memory blocks which a user can customize. **pm** and **dm** keywords will tell the linker to store these variables in program memory and data memory. SHARC-21489 internal memory is divided into 4 blocks; out of which two are 1.5 Mbits of size and other two are 1 Mbits in size.

Example

1. Case 1

```

#define SAMPLES 4800
dm int a[SAMPLES]
dm int b[SAMPLES];
dm int c[SAMPLES];

for (i=0; i<SAMPLES; i++)
  c[i] += a[i] * b[i];
  
```

To execute the above operation, it cost 7212 clock cycles (fcclk). Here, all the variables are stored in one data memory block.

2. Case 2

```
#define SAMPLES 4800
dm int a[SAMPLES]
dm int b[SAMPLES];
pm int c[SAMPLES];

for (i=0; i<SAMPLES; i++)
c[i] += a[i] * b[i];
```

To execute case 2, it cost 4818 clock cycles (*fclk*). Here, variables a & b are stored in one block and c will be stored in a different block and a and b will be accessed simultaneously.

For the above code, core will simultaneously take data from dm and pm and execute instruction.

Using Core & Hardware Accelerators simultaneously: FIR/FFT/IIR accelerators are designed to run faster for the large array of input data. And these accelerators can be used simultaneously with core which reduces load on the core by off-loading its tasks.

Accelerators run at half of system clock (*fpclock*), hence user has to write his filtering applications in such a way that the accelerator can filter the current samples and while utilizing core to perform other tasks.

Example

Let's take an example of filtering 1024 samples using 16 sub-band; each with 256 coefficients.

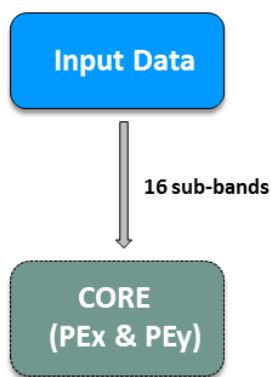


Fig. (a) 16 sub-bands processed by core

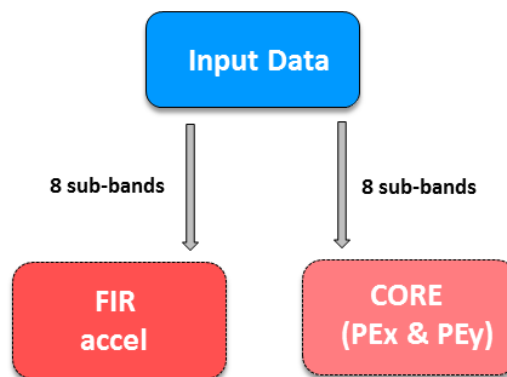


Fig. (b) 8 sub-bands processed by core and other 8 by FIR accelerator

1. Case 1 – Only Core

- In this case only core was used to filter data. Here for FIR filtering, highly optimized DSP library function **firf** was used
- The input data was filtered over 16 sub-bands one-by-one which consumed over **4 Million clock cycles (4252998) (fcclk)**

2. Case 2 – Using Core and Accelerator

- For the same requirement, if we go ahead with the core and accelerator simultaneously, we stand to gain nearly **2 Million Cycles (fcclk)**
- The total of 16 sub-bands were split equally between the core and accelerator to perform filtering
- Half of the sub-bands were processed by core and the other half by fir accelerator.
- This approach consumed nearly **2 Million cycles (2140899) (fcclk)** in total
- Cycles almost reached half. Thus, by using FIR_ACCEL+CORE effectively, the computational cycles were reduced

Using DMA:

- **Using External Memory:** Many DSP applications may require to operate on huge chunks of data which end up in memory constraints. If application uses any external data, then it takes numerous cycles for core to process that data directly. In such a case, it is recommended to copy all the data to an internal location and then process the internal memory data

Example

An application requires to transfer 1024 samples of data from external memory to internal memory which can be done in two ways:

1. Case 1

Using memcpy where core does the job of transferring data from external memory (sdram) to internal memory. In this case, core takes 23,399 clock cycles (fcclk).

2. Case 2

SHARC has DMA support from external to internal memory which transfers data in chunks(pages) from sdram to internal ram.

This transfer consumes 5307 clock cycles(fcclk). Also, the core is idle and can be used to execute other tasks as well.

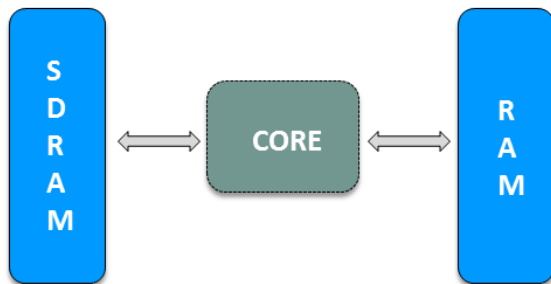


Fig. (a) core performing data transfer

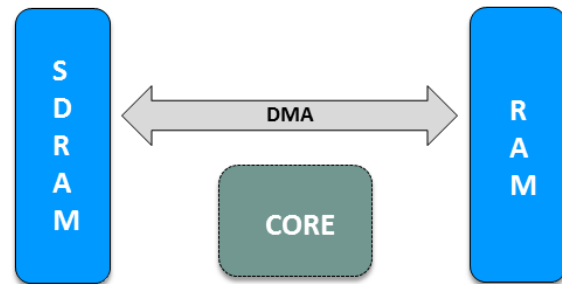


Fig. (b) DMA performing data transfer and core is idle

- **Using Memory-to-Memory (MTM) transfer:** In many applications; internal memory-to-internal memory copying is performed. In such a case, **MTM DMA (Memory-to-Memory DMA)** can be used to copy data internally so that core can perform other tasks.

Note: Such copying takes more clock cycles. For instance, transferring 62-bits of data requires 12 peripheral clock cycles (fpcclk).

Conclusion

There are many advantages in generating an efficient floating-point code by using different capabilities of the SHARC processors. Due to its mature floating-point architecture, even complex signal processing routines can fit in the desired cycles and memory budgets by writing an efficiently optimized code.

We implemented Acoustic Echo Cancellation Algorithms for echo cancellation both in time domain and frequency domain. In one of our sub-band based AEC algorithm, with the usage of FFT/FIR accelerators, intrinsics from DSP library and with certain changes to code flows along with compiler optimizations, we achieved optimization to the tune of 60x when compared to the previously recorded MCPS (Million Cycles Per Second) on SHARC-21489.

References

- [\[ADSP-214xx hardware reference\]](#)
- [\[ADSP -21489 Datasheet\]](#)
- [\[CCES Compiler manual for SHARC\]](#)
- [\[CCES C/C++ and DSP library manual for SHARC\]](#)

Learn more about PathPartner's product engineering services

More resources available at <https://www.pathpartnertech.com/insights/whitepapers/>
For an in-depth discussion, contact us at marcom@pathpartnertech.com

About PathPartner

PathPartner Technology is a leader in providing technology solutions and product engineering services for developing innovative intelligent systems. PathPartner offers a portfolio of enabling products and comprehensive services spanning embedded systems, hardware design, and system integration. We work with device manufacturers, system integrators and solution providers across diverse industry domains such as automotive, consumer electronics, drones, video surveillance, medical devices etc. Our deep technology expertise in multimedia systems, deep learning solutions, vision-based systems, connectivity technologies, and imaging pipelines coupled with our experience in underlying hardware platforms, enables us to provide complete solutions to our customers while expediting their time to market, and future proofing investments.

Headquartered in Bangalore, India, PathPartner has presence in USA, Europe and Japan. To learn more about us, please visit www.pathpartnertech.com.

Contact Us

PathPartner Technology Pvt. Ltd.

#16, PSS Plaza, 1st and 2nd Floor,
New Thippasandra Main Road,
HAL III Stage, Bangalore-560075.

Other presence: Bay Area, USA | Tokyo, Japan | Munich, Germany

Phone: +91 80 6772 2000 | +1 408 242 7411 | +81 905546681

Email: marcom@pathpartnertech.com

Website: www.pathpartnertech.com

Follow us on social media: [LinkedIn](#) | [Twitter](#)